Chapter 1
Fundamental of Operating System(OS)

Outline

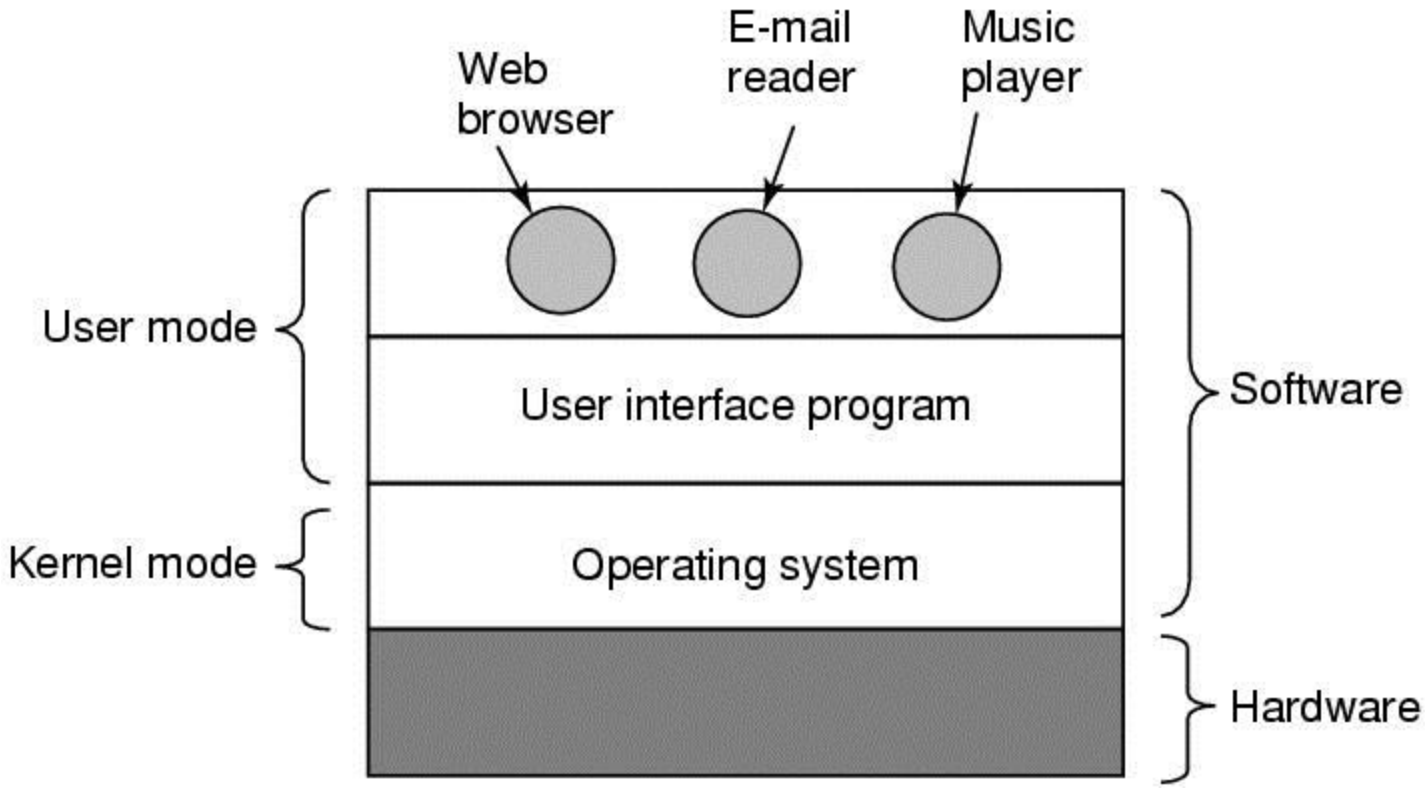
- Definition of OS
- Computer System Organization and Operation
- Operating System Operation
- Functions of OS
- Evolution of Operating Systems
- OS Services
- Types of OS
- System calls
- Kernel
  - Monolithic kernel
  - Microkernel
- Operating System structure
  - Simple Structure (MSDOS)
  - Layered Approach
  - Microkernel Approach
  - Modules Approach
  - Client Server Approach

Operating System Definition

- A program that acts as an intermediary between a user of a computer and the computer hardware

- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner

Operating System Definition



Where the operating system fits in.

Operating System Definition

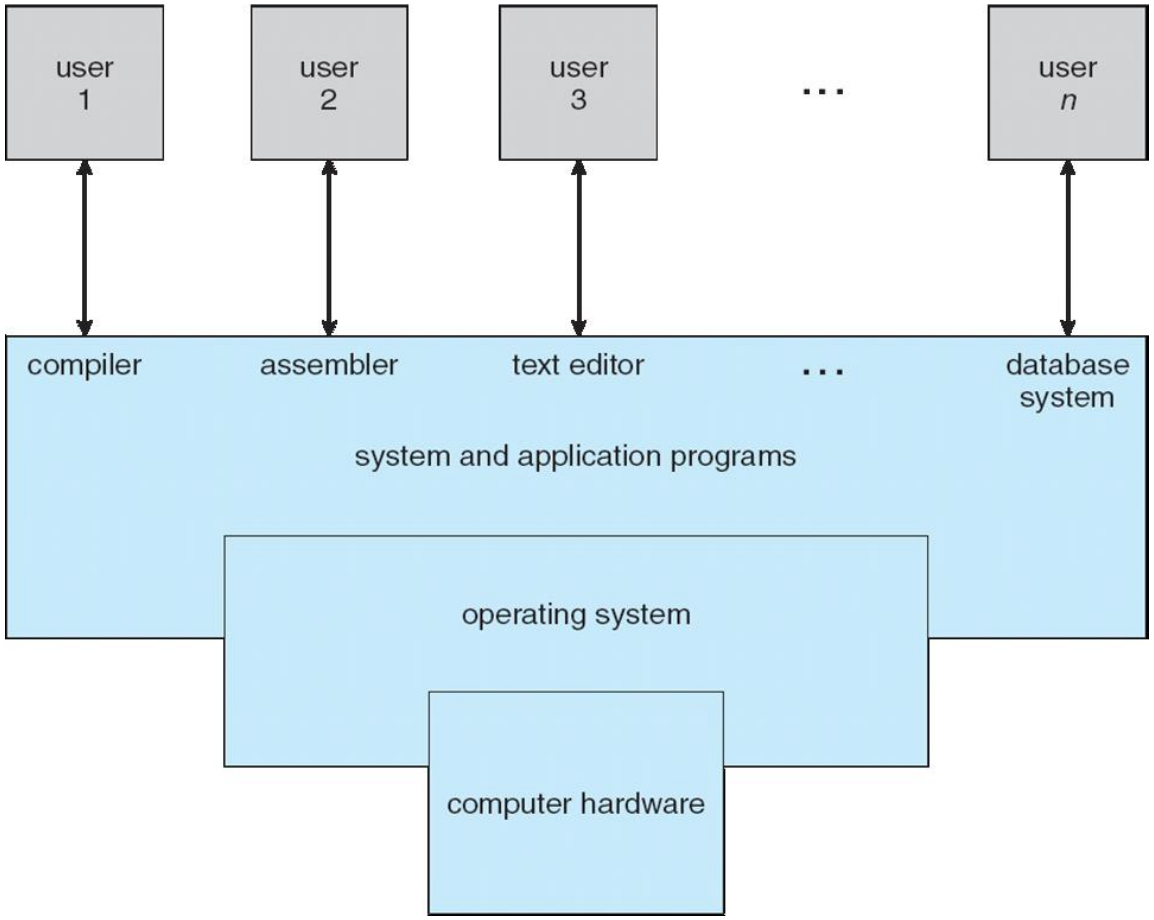
- No universally accepted definition
- “Everything a vendor ships when you order an operating system” is good approximation
  - But varies wildly
- “The one program running at all times on the computer” is the **kernel**. Everything else is either a system program (ships with the operating system) or an application program.

Computer System Organization and Operation

Computer System Structure

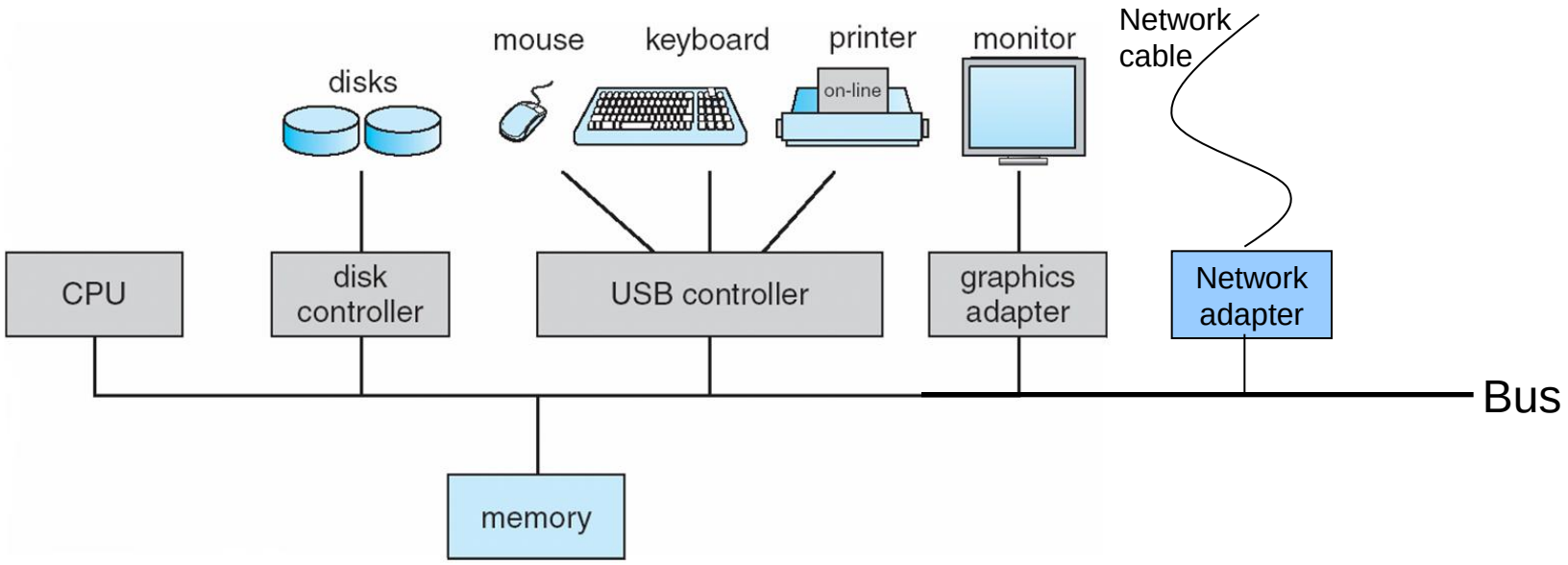
- Computer system can be divided into four components:
  - Hardware – provides basic computing resources
    - CPU, memory, I/O devices
  - Operating system
    - Controls and coordinates use of hardware among various applications and users
  - Application programs – define the ways in which the system resources are used to solve the computing problems of the users
    - Word processors, compilers, web browsers, database systems, video games
  - Users
    - People, machines, other computers

Four Components of a Computer System



Computer System Organization

- Computer-system operation
  - One or more **CPUs**, **device controllers** connect through common bus providing access to **shared memory**
  - Concurrent execution of CPUs and devices competing for memory cycles



Computer Startup

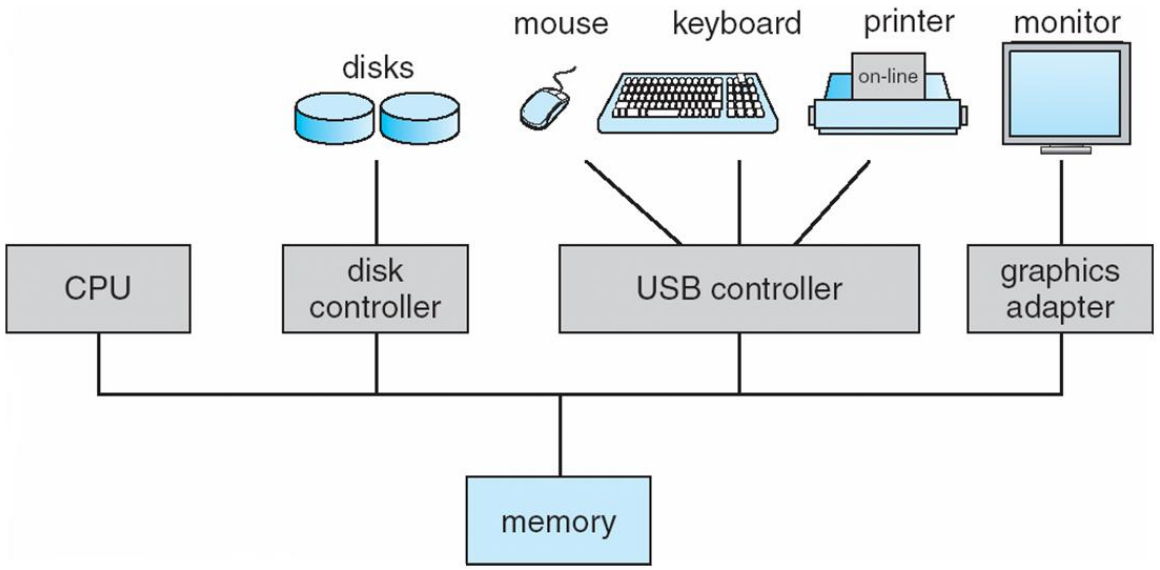
- **bootstrap program** is loaded at power-up or reboot
  - Typically stored in ROM or EPROM,
  - generally known as **firmware**
  - Initializes all aspects of the system
  - Loads operating system **kernel** and starts execution

- Kernel runs and make the system ready for running applications
  - Kernel is always ready to run (always in memory)

Computer system operation: I/O and device interaction

- I/O devices and the CPU can execute concurrently
- Each device controller has a local buffer
  - Data movement (I/O) between device and local buffer (device controller)
  - Data movement between memory and local buffer (by CPU)

- Device controller informs CPU that it has finished its operation by causing an *interrupt*



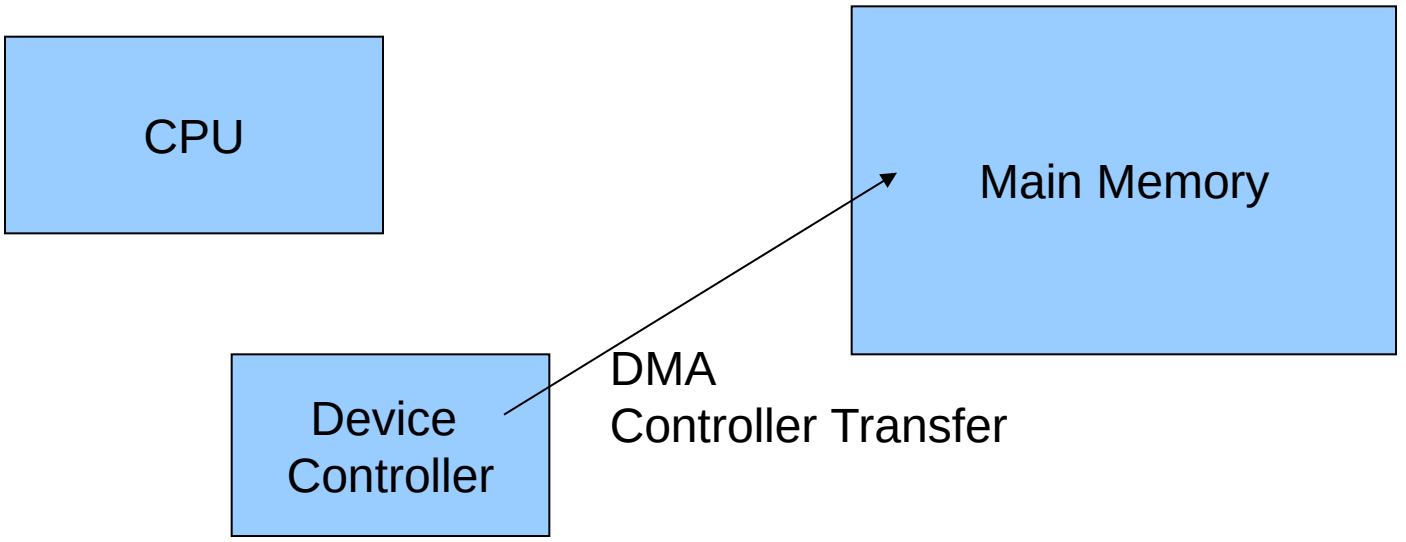
Hardware interrupts

- When interrupt occurs, hardware does the following:
  - CPU is interrupted
    - at that time application code or kernel code might be running
    - registers and the program counter saved in RAM to preserve CPU state
    - CPU starts running the respective Interrupt Service Routing (ISR)
      - (kernel routine)
  - ISR is found through **interrupt vector**
    - (table containing addresses of ISRs)

- Incoming interrupts are *disabled* while an interrupt is being processed to prevent a *lost interrupt.*

Direct Memory Access Structure

- Used for **high-speed** I/O devices able to transmit information at close to memory speeds
- Device controller **transfers blocks of data** from buffer storage directly to main memory **without CPU intervention**
- Only **one interrupt is generated per block**, rather than the one interrupt per byte

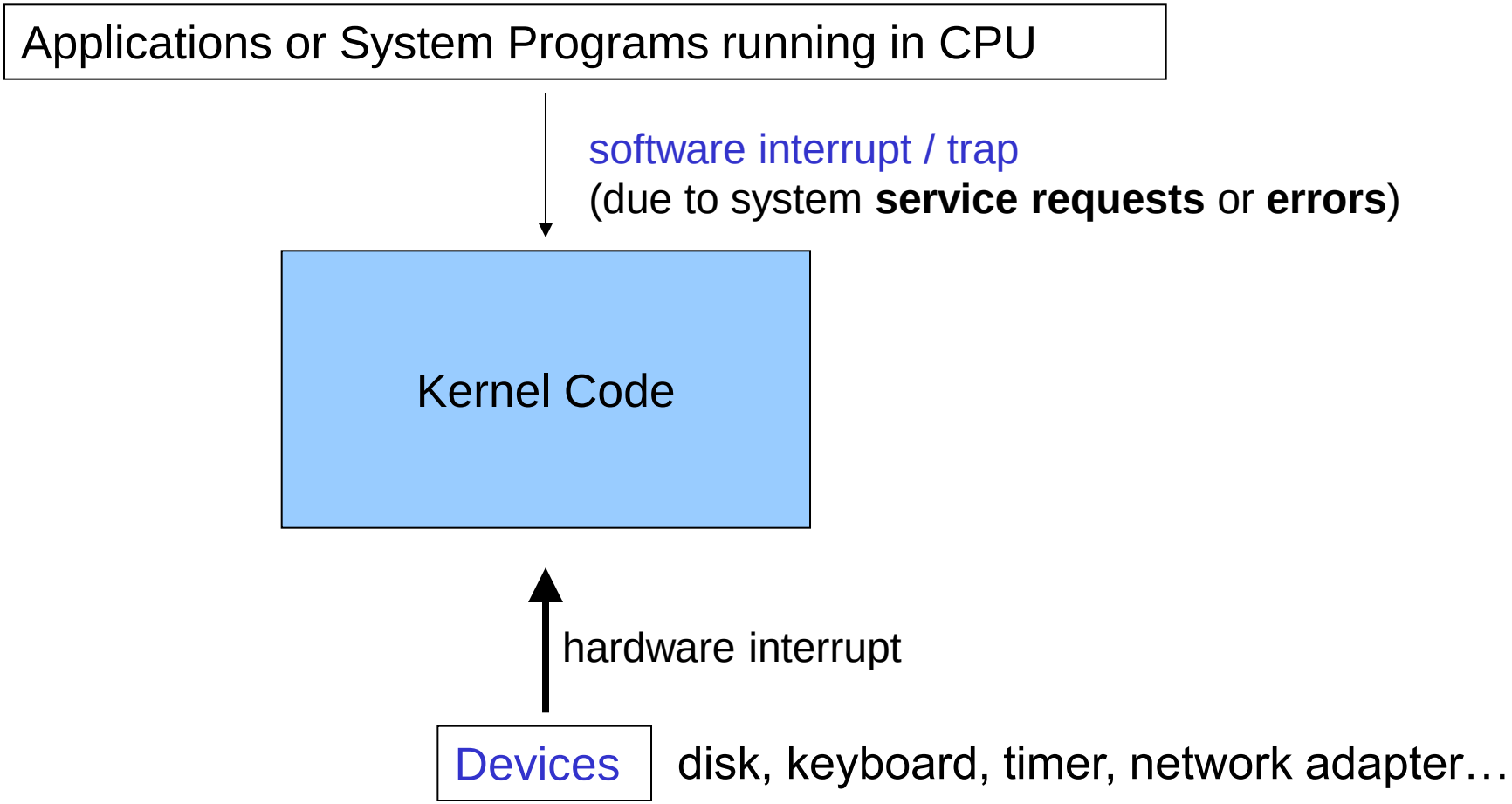


Software interrupts

- Running application software may generate interrupts as well.
  - They are called software interrupts (also called traps)
    - 1. exceptions (caused by errors)
    - 2. system calls (service request)
      - **trap** or **syscall** instruction is used

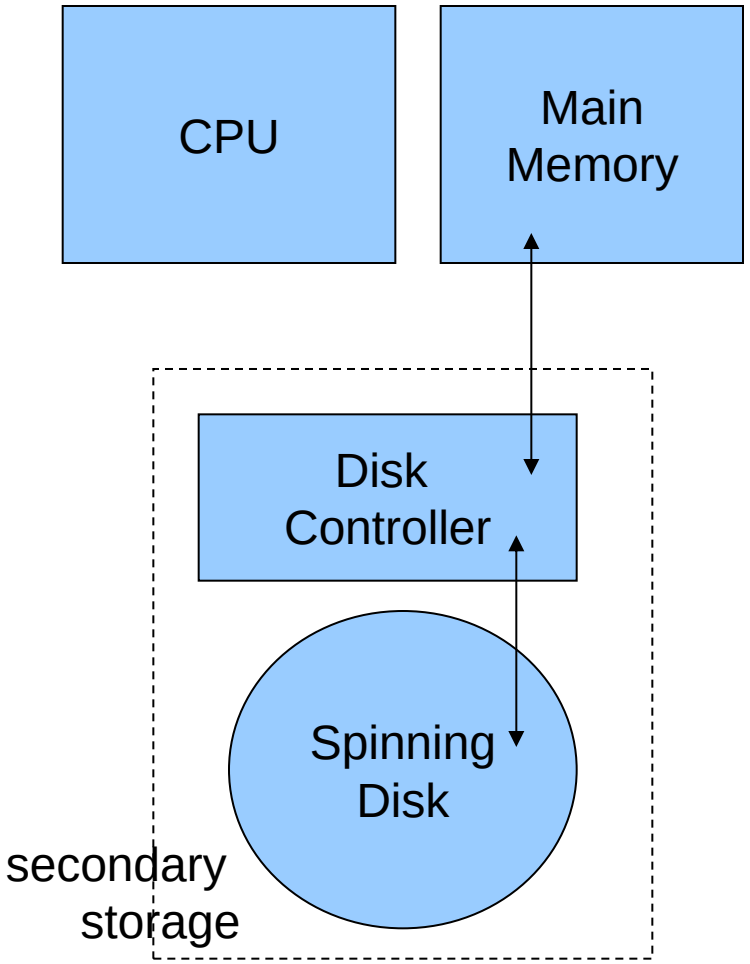
- **An operating system (kernel) is interrupt-driven (event driven)**

Interrupt-Driven OS



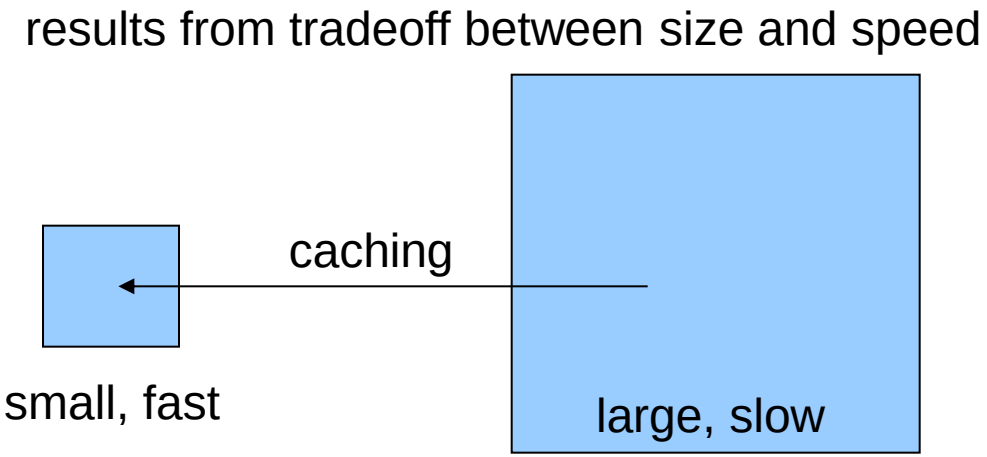
Storage Structure

- **Main memory** – only large storage media that the CPU can access directly
- **Secondary storage** – extension of main memory that provides large nonvolatile storage capacity
  - Magnetic disks
    - Rigid metal or glass platters covered with magnetic recording material
    - The **disk controller** determines the logical interaction between the device and the computer

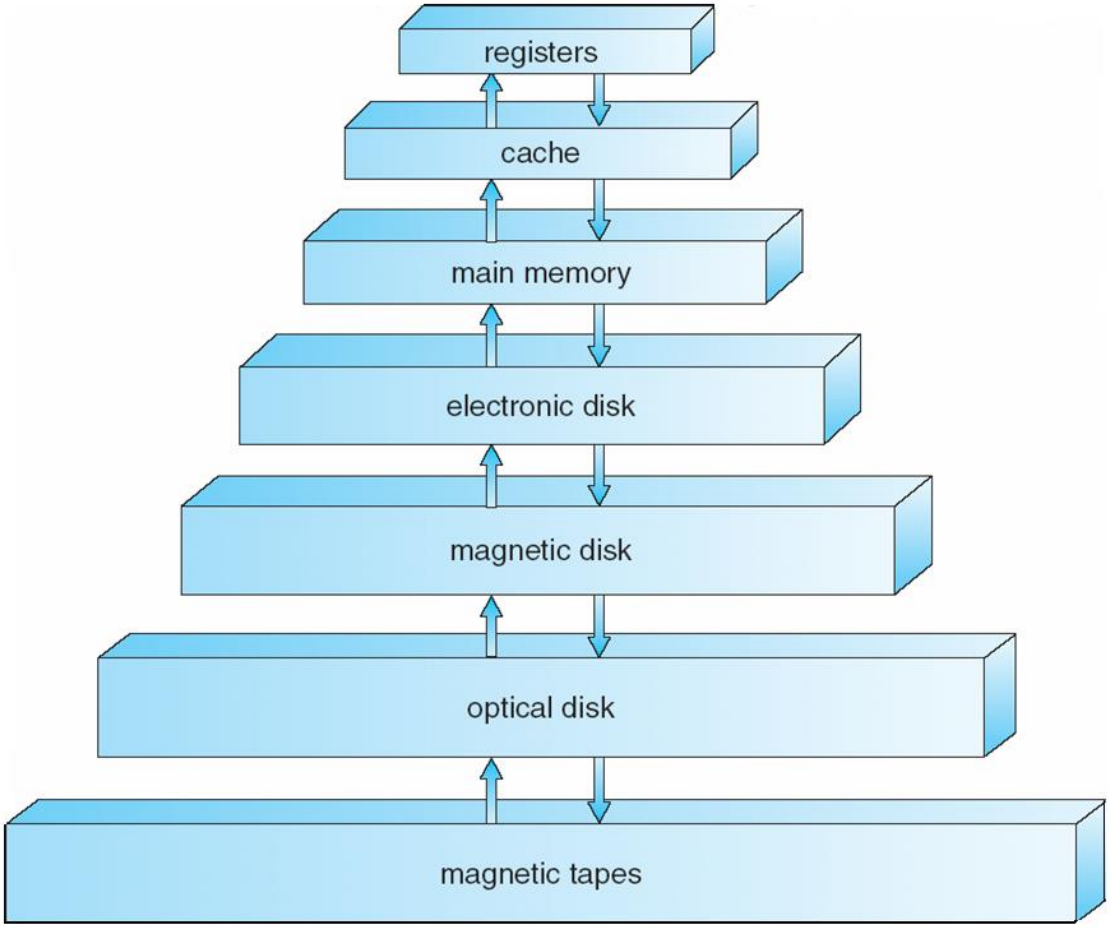


Storage Hierarchy

- Storage systems **organized in hierarchy**
  - Speed
  - Cost
  - Volatility
- **Caching** – copying information into faster storage system; main memory can be viewed as a last *cache* for secondary storage



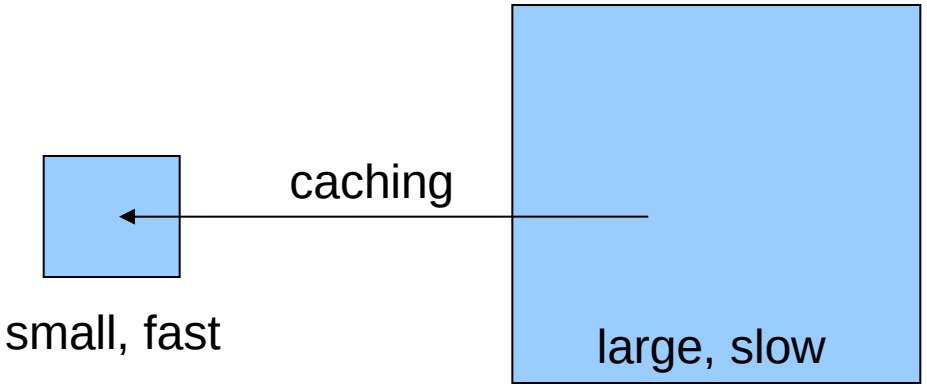
Storage-Device Hierarchy



Caching

- Important principle, performed at **many levels** in a computer (in hardware, operating system, software)
- Information in use **copied from slower to faster storage** temporarily
- Faster storage (cache) checked first to determine if information is there
  - If it is, information used directly from the cache (fast)
  - If not, data copied to cache and used there

Caching

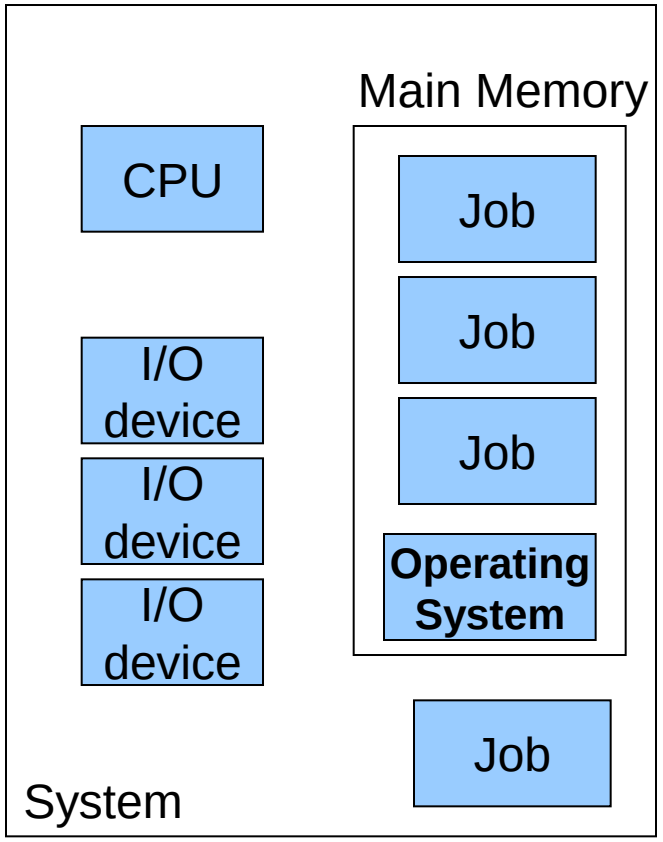


Registers	Main Memory
Hardware cache L1, L2, etc	Main Memory
Main Memory	Hard Disk
Hard Disk	Tape

Operating System Operation

Operating Systems: providing multiprogramming

- **Multiprogramming** needed for **efficiency**
  - Single user cannot keep CPU and I/O devices **busy** at all times
  - Multiprogramming organizes **jobs** (code and data) so CPU always has one to execute
  - A subset of total jobs in system is kept in memory
  - One job selected and run via **job scheduling**
    - OS selects which job
  - When it has to wait (for I/O for example), OS switches to another job

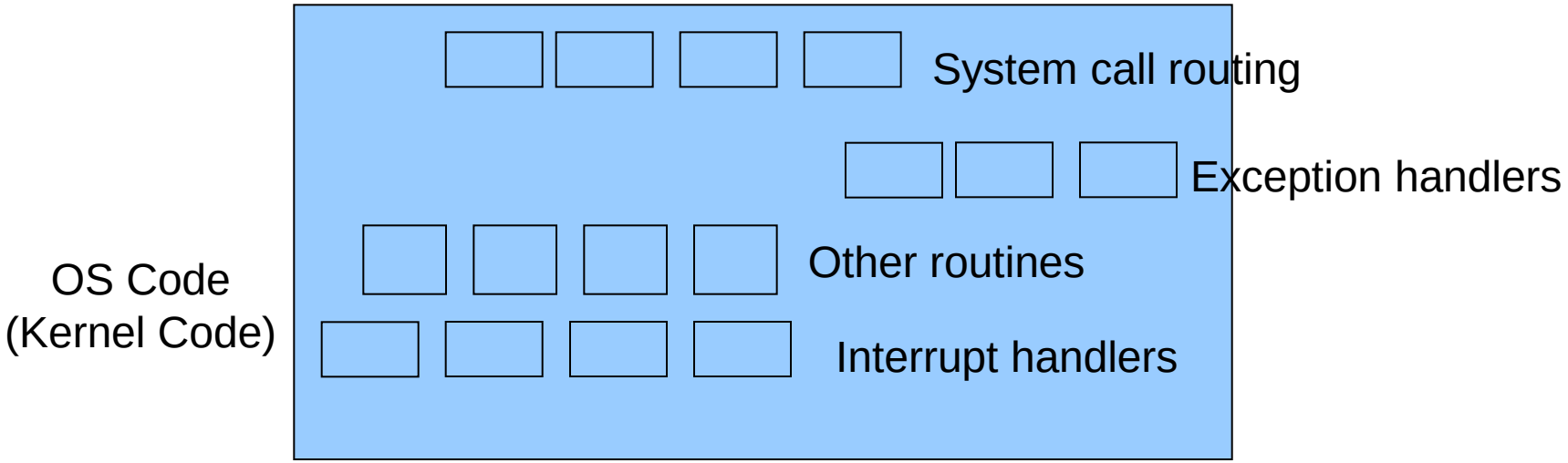


Operating Systems: providing time sharing

- **Timesharing (multitasking)** is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing
  - **Response time** should be < 1 second
  - program loaded in memory ⇨**process**
  - If several processes ready to run at the same time ⇨ **CPU scheduling**

Operating System: how operates

- is interrupt driven
  - Hardware interrupt causes ISR to run (which is a routine of OS)
  - Software error or request creates **exception** or **trap**
    - Division by zero, for example (exception)
    - request for an operating system service (trap)



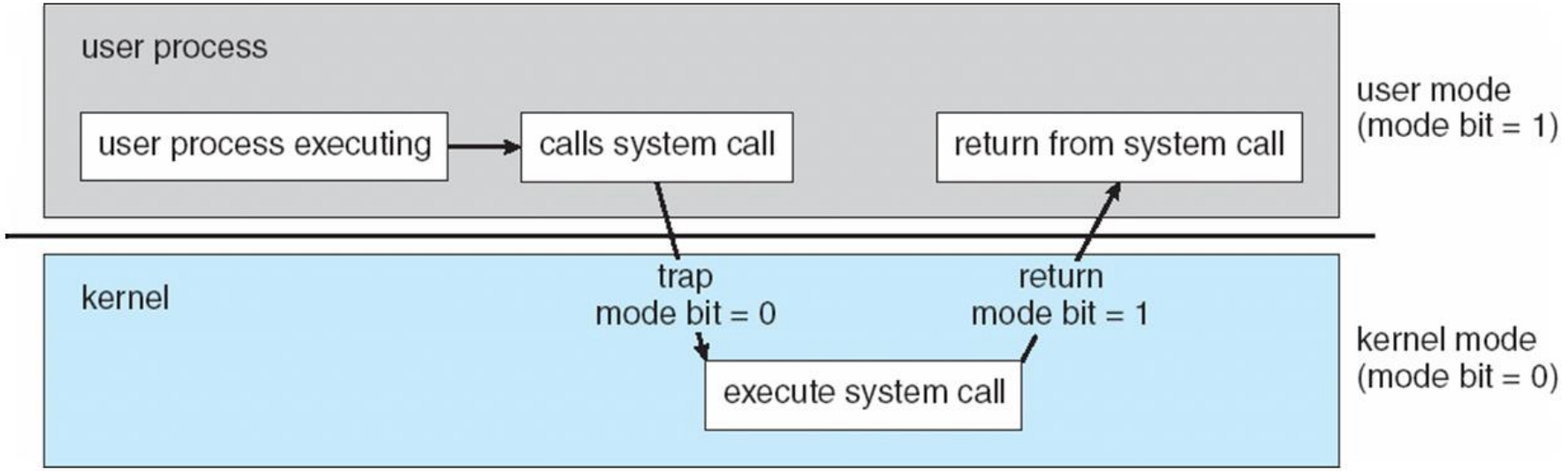
Operating System: how operates

- **Dual-mode** operation allows OS to protect itself and other system components
  - **User mode** and **kernel mode**
  - **Mode bit** provided by hardware
    - user code or kernel code in different modes
    - Some machine instructions designated as **privileged**, only executable in kernel mode
    - System call changes mode to kernel, return from call resets it to user mode

Operating System: how operates

Dual mode system operation

Transition from User to Kernel Mode and Vice Versa



Operating System: how operates

- **Timer device** to prevent infinite loop / process hogging resources
  1) Set the timer device to interrupt after a while
     Can be a fixed or variable time period
  2) CPU executes a program (a process)
  3) Timer device sends an interrupt after that period
  4) CPU starts executing timer handler: OS gains control
  5) OS can schedule the same process or other process
  6) OS sets the timer again before giving the CPU to the scheduled process

Functions of OS

Functions of OS

- Booting the computer
- Performs basic computer tasks eg managing the various peripheral devices eg mouse, keyboard
- Provides a user interface, e.g. command line, graphical user interface (GUI)
- Handles system resources such as computer's memory and sharing of the central processing unit (CPU) time by various applications or peripheral devices
- Provides file management which refers to the way that the operating system manipulates, stores, retrieves and saves data.

Functions of OS

Following are some of important functions of an operating System.

- Process Management
- Memory Management
- I/O Management
- Storage Management (File Management)
- Protection and Security
- Control over system performance
- Job accounting
- Coordination between other software and users

Process Management

- **A process is a program in execution**.
  - **Unit of work** in the system
  - Process is an *active entity* (a program is passive).
- Process executes instructions **sequentially**, one at a time, until completion
- Process **needs resources** to accomplish its task
  - CPU, memory, I/O, files
- Typically system has **many processes** running **concurrently**
  - Some of them may be OS processes
- Upon termination, **resources are released**

For process management:

- **Creating** and **deleting** both user and system processes and
- **Suspending resuming** processes
- Providing mechanisms for process **synchronization**
- Providing mechanisms for process **communication**
- Providing mechanisms for **deadlock** handling

Memory Management

- All data in memory before and after processing
- All instructions in memory in order to execute
- Memory management determines what is in memory, where and when
- Memory management activities
  - **Keeping track** of which parts of memory are currently being used and by whom
  - Deciding **which processes** (or parts of a process) and data to move into and out of memory
  - **Allocating** and **deallocating** memory space as needed

Performance of various levels of storage

- Movement between levels of storage hierarchy can be explicit or implicit.

Level	1	2	3	4
Name	registers	cache	main memory	disk storage
Typical size	< 1 KB	> 16 MB	> 16 GB	> 100 GB
Implementation technology	custom memory with multiple ports, CMOS	on-chip or off-chip CMOS SRAM	CMOS DRAM	magnetic disk
Access time (ns)	0.25 – 0.5	0.5 – 25	80 – 250	5,000.000
Bandwidth (MB/sec)	20,000 – 100,000	5000 – 10,000	1000 – 5000	20 – 150
Managed by	compiler	hardware	operating system	operating system
Backed by	cache	main memory	disk	CD or tape

Storage Management

- OS provides **uniform**, **logical view of information storage**
  - Abstracts physical properties to **logical storage unit - file**
    - **Various storage types** varying in medium type, access speed, capacity, data-transfer rate, access method

- **File-System management**
  - Files usually organized into **directories**
  - **Access control** on most systems to determine who can access what

- OS activities include
    - **Creating** and **deleting files** and **directories**;
    - Primitives to **manipulate files/dirs**;
    - **Mapping files onto secondary storage**

Mass-Storage Management

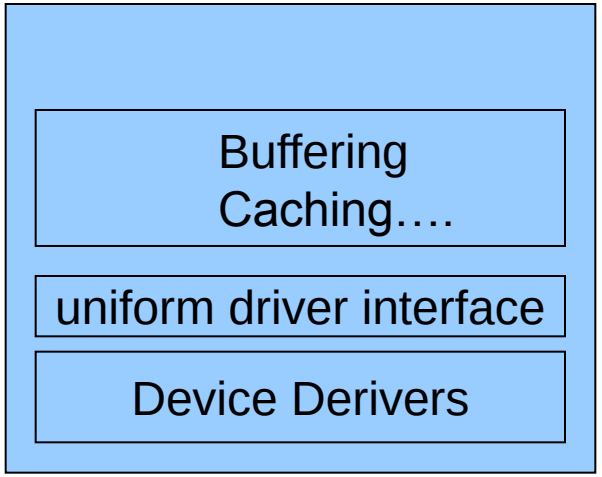
- Mass Storage:
  disk (secondary);
  tapes,
  CDs, etc. (tertiary)

- Proper management of mass storage devices is of central importance
  - For improving **performance** of the computer system
  - Since they are **slow devices**

- OS activities
  - **Free-space management**; **Storage allocation**
  - **Disk scheduling**
  - **Uniform naming ….**

Input/Output Subsystem

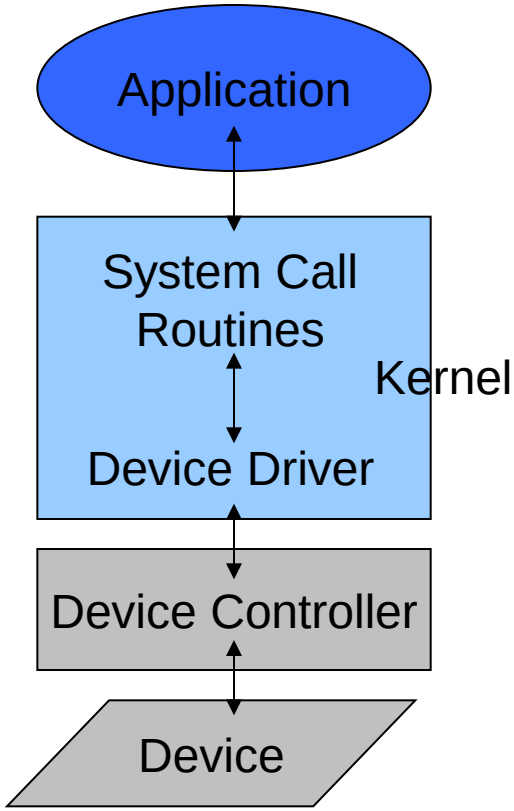
- One purpose of OS is to **hide peculiarities of hardware** devices from the user
- **I/O subsystem responsible for**
  - **Buffering**, **caching**
  - General **device-driver interface**
  - **Drivers** for specific hardware devices
    - **Interacting with the device and doing I/O**



I/Os sub-system of Kernel

I/O Structure

- Application programs do I/O via OS
  - The request is done by calling a System Call (OS routine)
  - System call routine in OS performs the I/O via the help of device driver routines in OS.
  - After issuing a system call, an application may wait for the call to finish (blocking call) or may continue to do something else (non-blocking call)



Input/Output Management

OS activities

- Keeps tracks of all devices. Program responsible for this task is known as the I/O controller.
- Decides which process gets the device when and for how much time.
- Allocates the device in the efficient way.
- De-allocates devices.

Protection and Security

- **Protection** – any mechanism for controlling access of processes or users to resources defined by the OS

- **Security** – defense of the system against internal and external attacks
  - Huge range, including denial-of-service, worms, viruses, identity theft, theft of service
- Systems generally first distinguish among users, to determine who can do what
  - User identities (**user IDs**, security IDs) include name and associated number, one per user
  - User ID then associated with all files, processes of that user to determine access control

Other Important Functions

Following are some of the important function that Operating System does.

- **Control over system performance --** Recording delays between request for a service and response from the system.

- **Job accounting --** Keeping track of time and resources used by various jobs and users.

- **Coordination between other software and users --** Coordination and assignment of compilers, interpreters, assemblers and other software to the various users of the computer systems.

Evolution of Operating Systems

Generations of Operating Systems

Generations:

- (1945–55) Vacuum Tubes
- (1955–65) Transistors and Batch Systems
- (1965–1980) ICs and Multiprogramming
- (1980–Present) Personal Computers, Tablets, Phones

Transistors and Batch Systems (1)

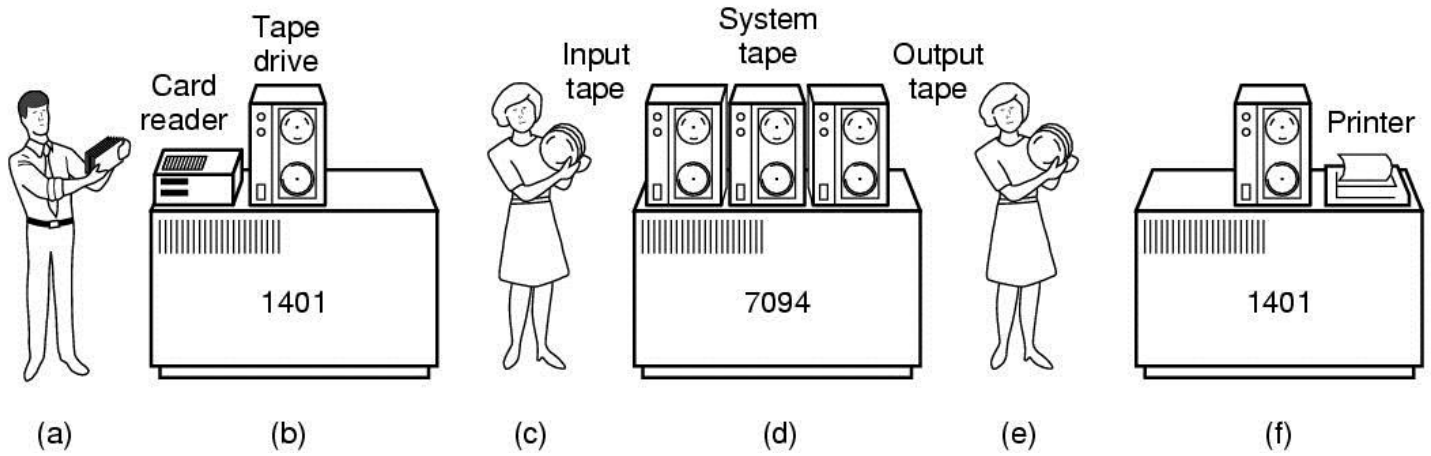
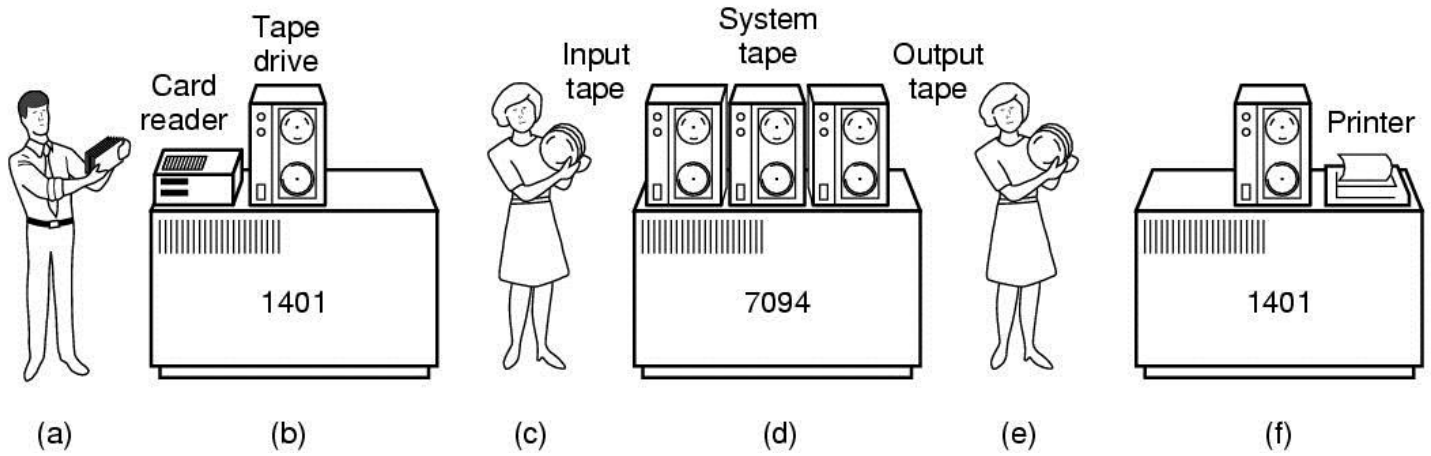


Figure 1-3. An early batch system.
(a) Programmers bring cards to 1401.
(b)1401 reads batch of jobs onto tape.

Transistors and Batch Systems (2)



(c) Operator carries input tape to 7094. (d) 7094 does computing.
(e) Operator carries output tape to 1401. (f) 1401 prints output.

Third and Fourth generation

Third generation

- Time Sharing
- Multics
- Unix
- Linux

Fourth generation

- PCs
- Network Operating Systems
- Distributed Operating Systems

Operating System Services

Operating System Services

- Operating systems provide an environment for execution of programs and services to programs and users
- One set of operating-system services provides functions that are helpful to the user:
  - **User interface** - Almost all operating systems have a user interface (UI).
    - Varies between **Command-Line (CLI)**, **Graphics User Interface (GUI)**, **Batch**
  - **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
  - **I/O operations** - A running program may require I/O, which may involve a file or an I/O device
  - **File-system manipulation** - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.

Operating System Services (Cont.)

- **Communications** – Processes may exchange information, on the same computer or between computers over a network
  - Communications may be via shared memory or through message passing (packets moved by the OS)
- **Error detection** – OS needs to be constantly aware of possible errors
  - May occur in the CPU and memory hardware, in I/O devices, in user program
  - For each type of error, OS should take the appropriate action to ensure correct and consistent computing
  - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

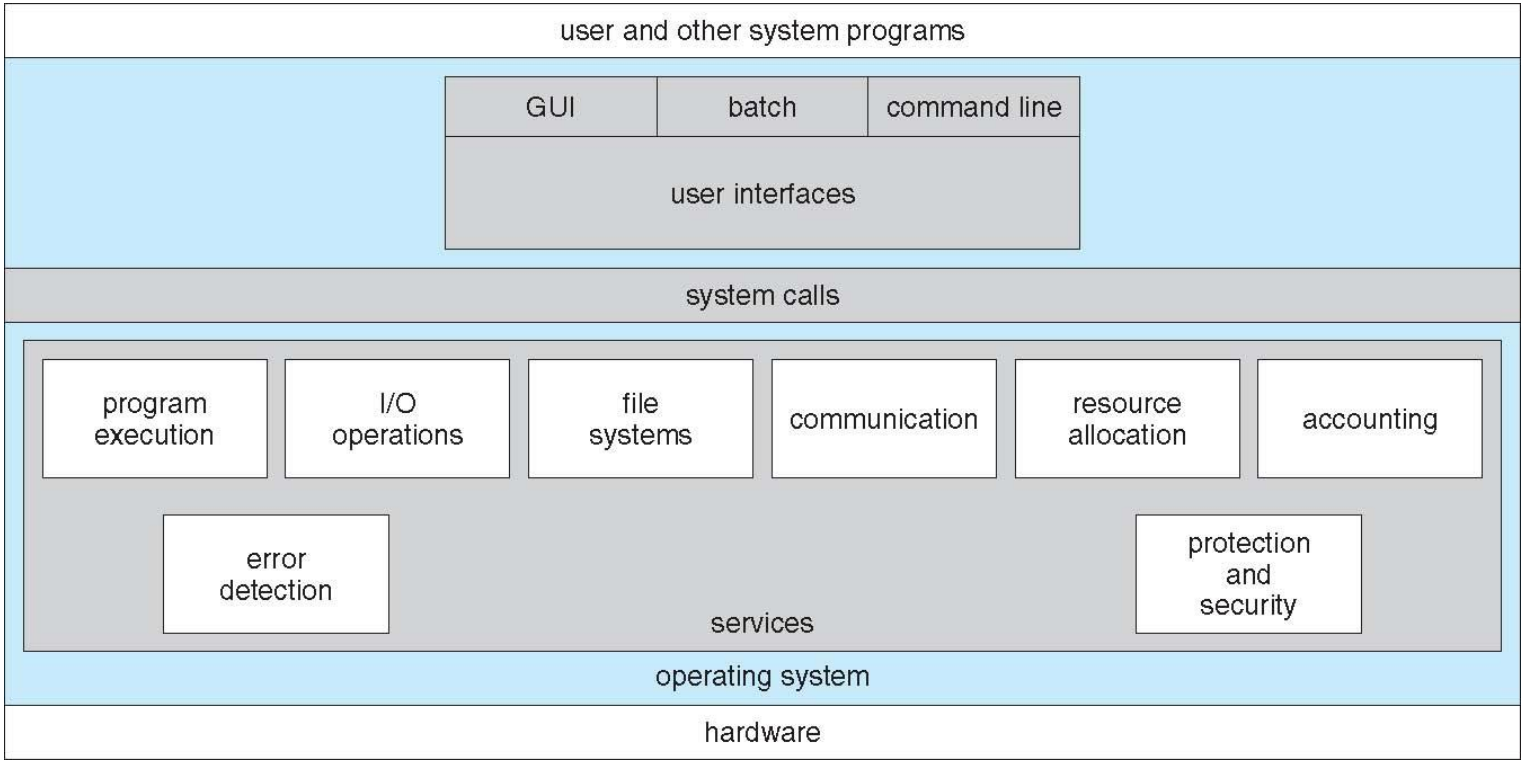
Operating System Services (Cont.)

- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing
  - **Resource allocation -** When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
    - Many types of resources - Some (such as CPU cycles, main memory, and file storage) may have special allocation code, others (such as I/O devices) may have general request and release code
  - **Accounting -** To keep track of which users use how much and what kinds of computer resources.

Operating System Services (Cont.)

- **Protection and security -** The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
  - **Protection** involves ensuring that all access to system resources is controlled
  - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts
  - If a system is to be protected and secure, precautions must be instituted throughout it. A chain is only as strong as its weakest link.

A View of Operating System Services



User Operating System Interface - CLI

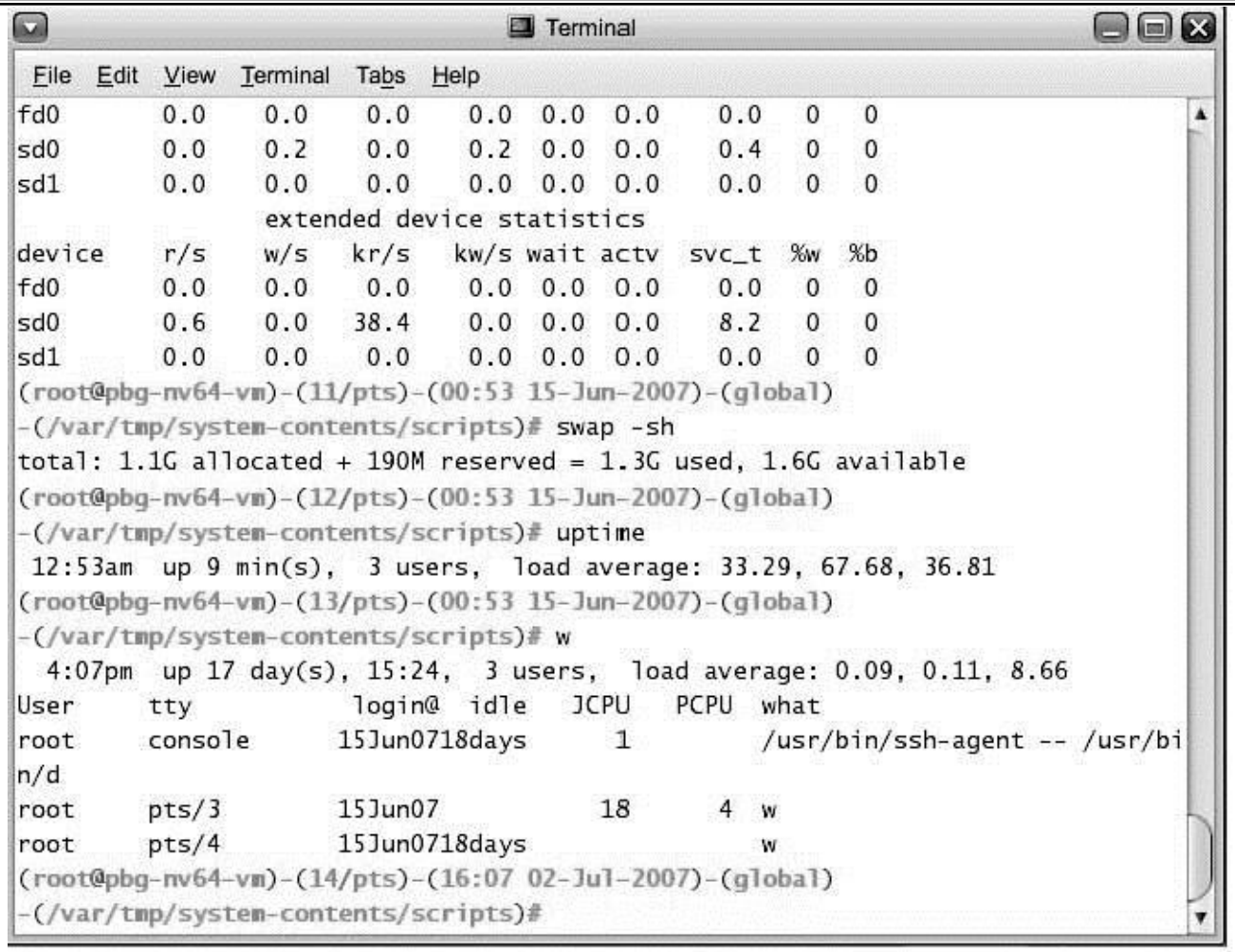
- Command Line Interface (CLI) or **command interpreter** allows direct command entry
  - Sometimes implemented in kernel, sometimes by systems program
  - Sometimes multiple flavors implemented – **shells**
  - Primarily fetches a command from user and executes it
    - Sometimes commands built-in, sometimes just names of programs
      - » If the latter, adding new features doesn't require shell modification

User Operating System Interface - GUI

- User-friendly **desktop** metaphor interface
  - Usually mouse, keyboard, and monitor
  - **Icons** represent files, programs, actions, etc
  - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a **folder**)
  - **Invented at Xerox PARC (**Palo Alto Research Center Incorporated**)**

- Many systems now include both CLI and GUI interfaces
  - Microsoft Windows is GUI with CLI “command” shell
  - Apple Mac OS X as “Aqua” GUI interface with UNIX kernel underneath and shells available
  - Solaris is CLI with optional GUI interfaces (Java Desktop, KDE)

Bourne Shell Command Interpreter



The Mac OS X GUI



Types of Operating System

Types of OS

- Multiprocessor operating systems
  - Use multiple cores
- Mainframe operating systems
  - Big-thousands of disks….
  - Lots of jobs with lots of I/O
  - Services-batch (payroll) transactions (airline reservations, timesharing (query database)
  - Elderly-Unix, Linux replacing them
- Server operating systems
  - Workstations
  - File, print, web servers
  - BSD, Linux, Windows

Types of OS

- Embedded operating systems-TV sets, cars, DVDs, MP3s
  - Everything is in ROM (no apps can run on it)
  - QNx, Vxworks
- Real time operating systems
  - Hard (eg. factory) deadline
  - Soft (eg. multi-media) deadline

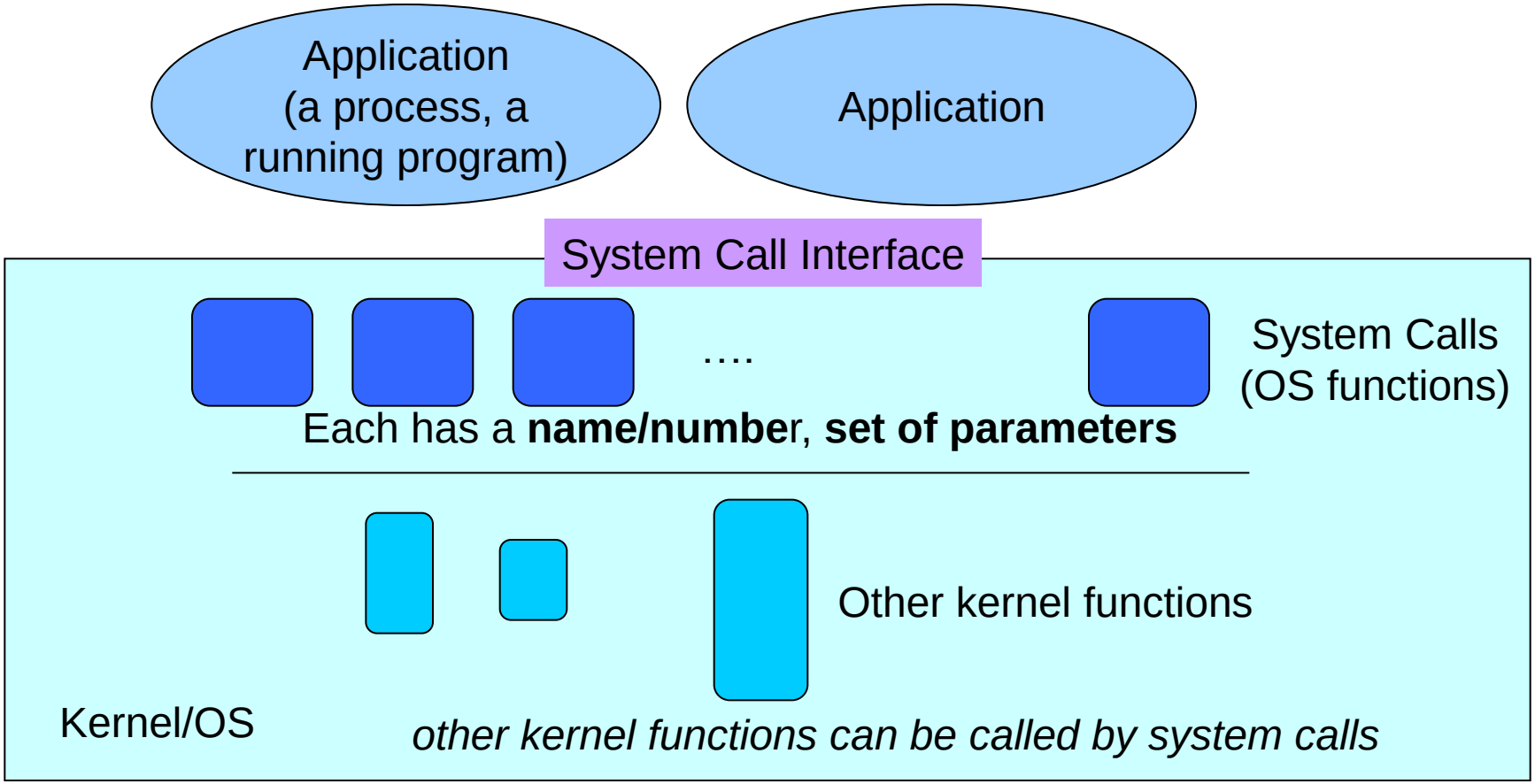
  Smart card OS (eg border crossing cards)
    - Java in ROM
- PC operating systems-Linux, Mac, Windows
- Smart phone operating systems- Android, iPhone, Blackberry
  - No hard disk
  - Palm, Symbian popular OS's

System Calls

System Calls

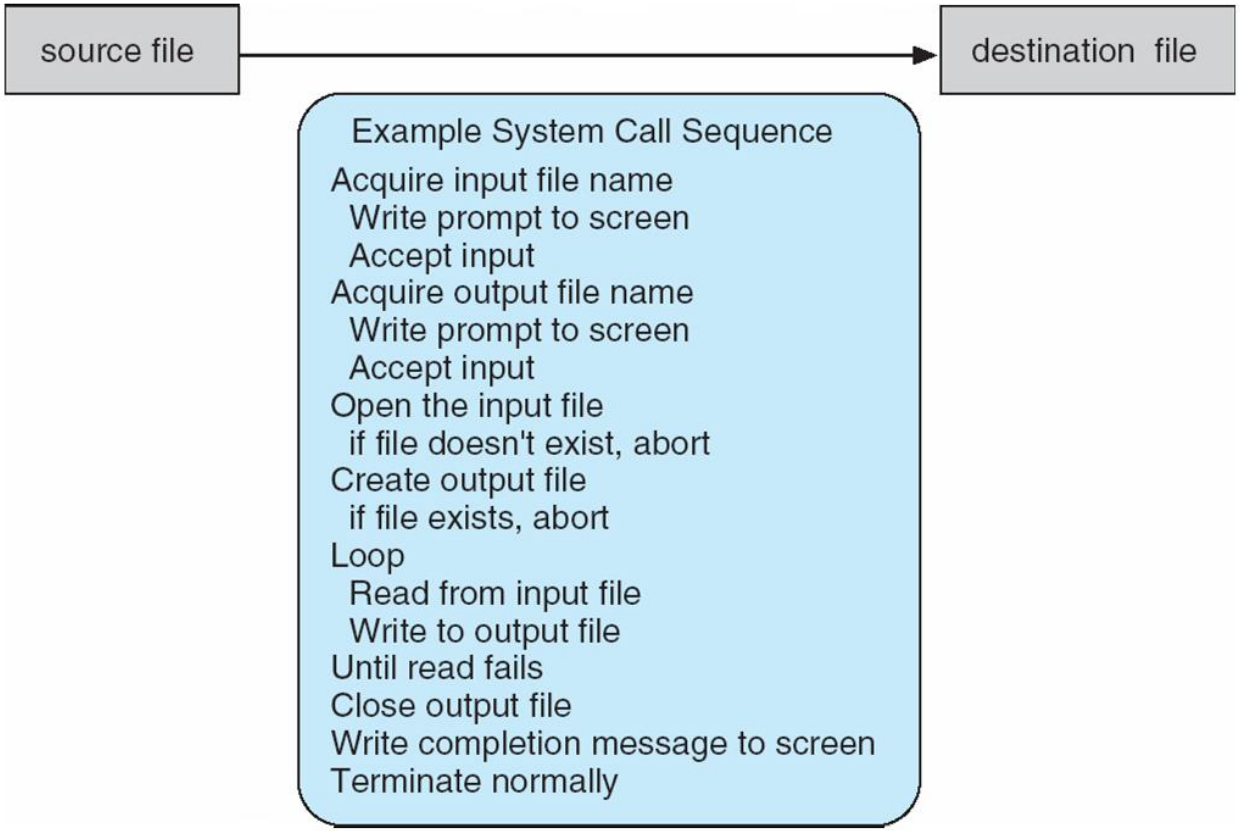
- Programming interface to the services provided by the OS
  - i.e. interface provided to applications
- Typically written in a high-level language (C or C++)
- Are called by a running program to get services
- Even a simple program may make a lot of calls per second.

System Calls



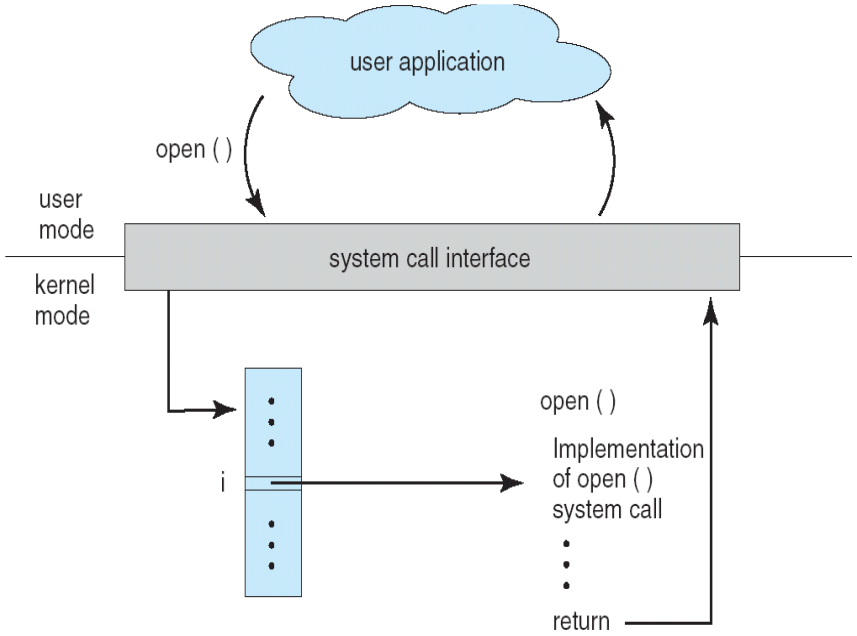
Example of System Calls

- System call sequence to copy the contents of one file to another file



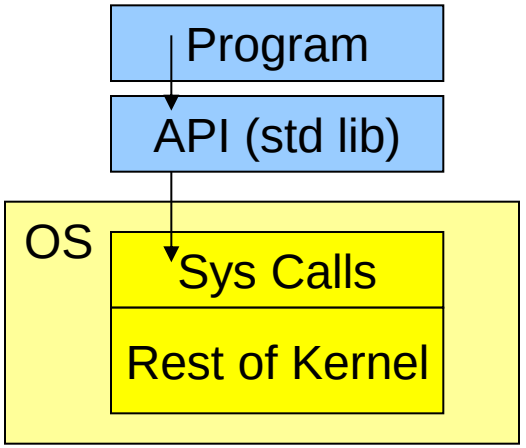
System Call Implementation and Calling

- Typically,
  - a **number** associated with each system call
  - Number used as an index to a table: System Call table
  - Table keeps addresses of system calls
  - System calls run and returns
- Caller does not know system call implementation
  - Just should know interface



Accessing and executing System Calls

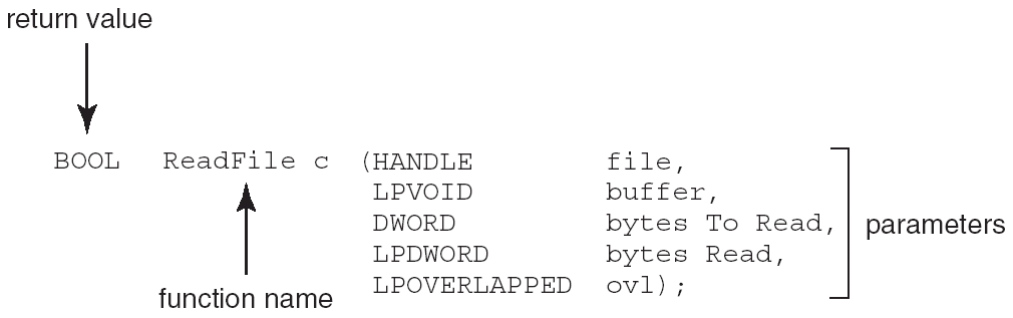
- System calls typically **not accessed directly** by programs
- Mostly accessed by programs via a high-level Application Program Interface (API) (i.e. a library) rather than direct system call use



- Three most common APIs are :
  - Win32 API for Windows,
  - POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X),
  - Java API for the Java virtual machine (JVM)

Example of Standard API

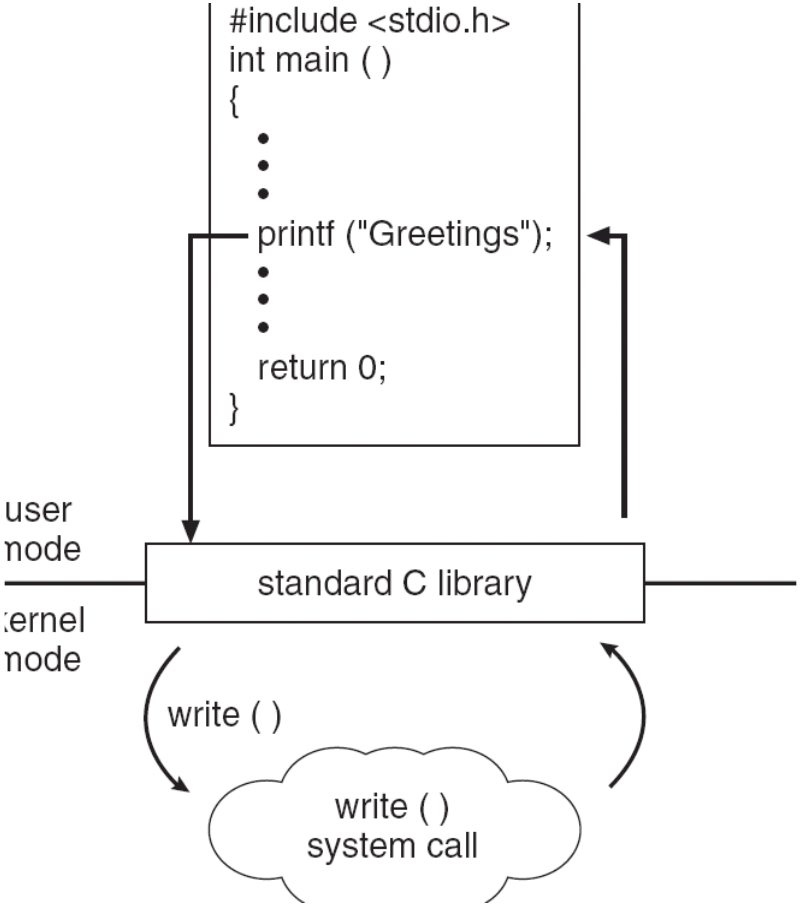
- Consider the ReadFile() function in the Win32 API—a function for reading from a file



- A description of the parameters passed to ReadFile()
  - HANDLE file—the file to be read
  - LPVOID buffer—a buffer where the data will be read into and written from
  - DWORD bytesToRead—the number of bytes to be read into the buffer
  - LPDWORD bytesRead—the number of bytes read during the last read
  - LPOVERLAPPED ovl—indicates if overlapped I/O is being used

Standard C Library Example

- C program invoking printf() library call, which calls write() system call

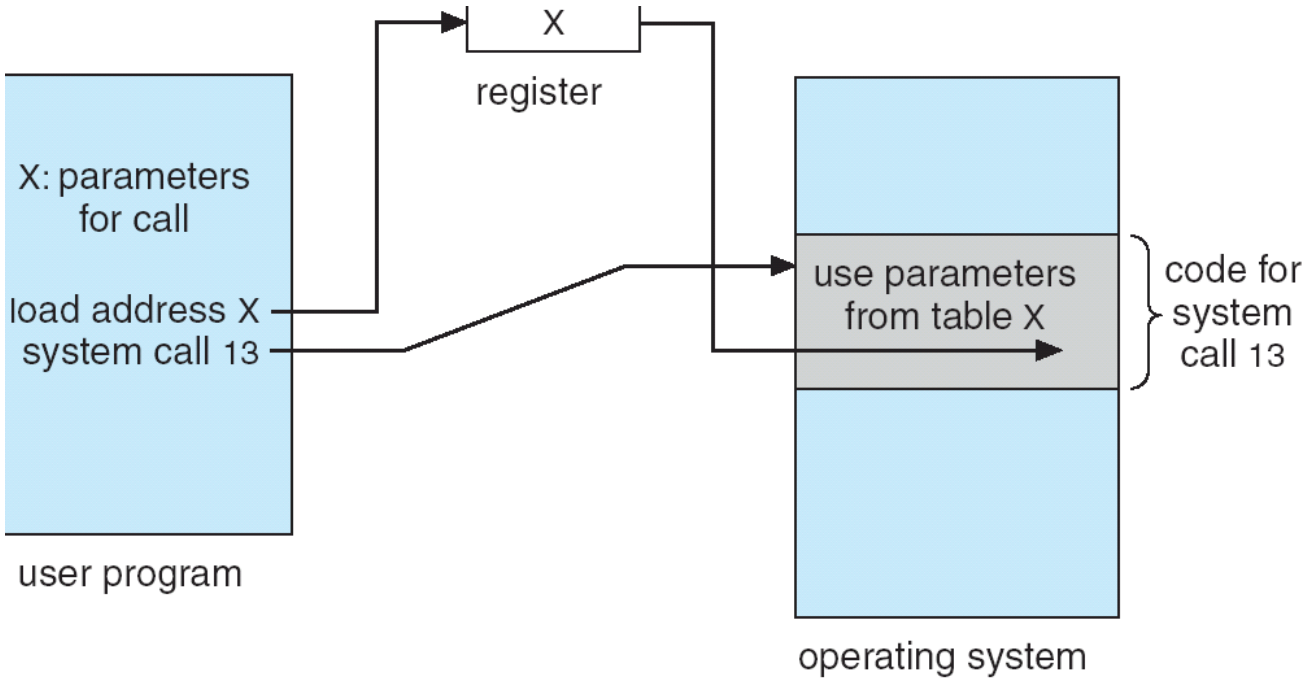


System Call Parameter Passing

- Often, more information is required than the identity of the desired system call
  - Exact type and amount of information vary according to OS and call
- Three general methods used to pass parameters to the OS
  - 1) Simplest: pass the parameters in *registers*
    - In some cases, may be more parameters than registers
  - 2) Parameters stored in a *block,* or table, in memory, and address of block passed as a parameter in a register
  - 3) Parameters placed, or *pushed,* onto the *stack* by the program and *popped* off the stack by the operating system

  Last two methods do not limit the number or length of parameters being passed

Parameter Passing via Table



Types of System Calls

- **Process control**
  - end, abort
  - load, execute
  - create process, terminate process
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory
- **File management**
  - create file, delete file
  - open, close file
  - read, write, reposition
  - get and set file attributes

Types of System Calls (Cont.)

- Device management
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices
- Information maintenance
  - get time or date, set time or date
  - get system data, set system data
  - get and set process, file, or device attributes
- Communications
  - create, delete communication connection
  - send, receive messages
  - transfer status information
  - attach and detach remote devices

Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Kernels

Introduction to Kernels

- Different Types of Kernel Designs
  - Monolithic kernel
  - Microkernel
  - Hybrid Kernel
  - Exokernel

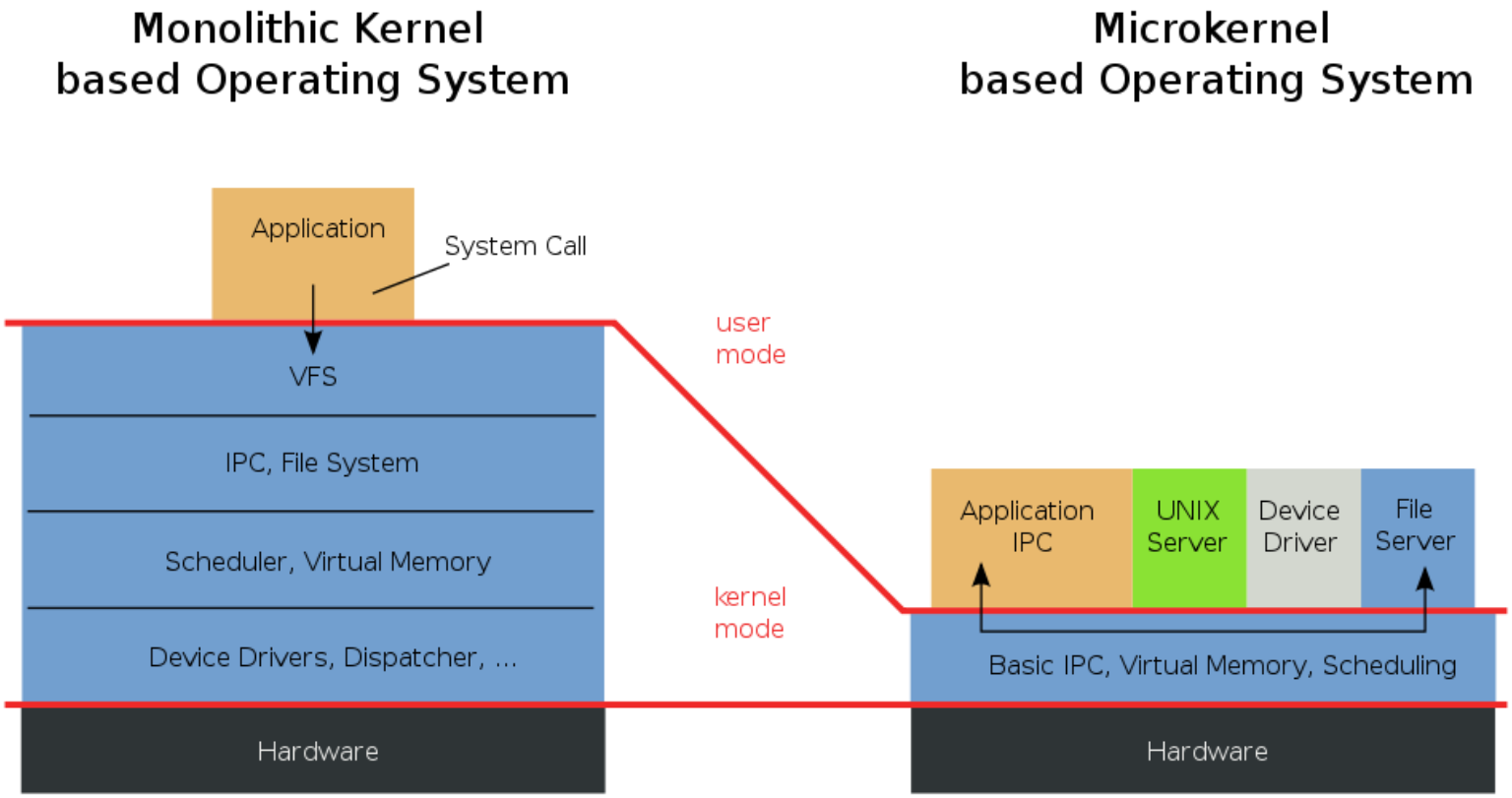
Monolithic Kernels

- All OS services operate in kernel space
- Good performance
- Disadvantages
  - Dependencies between system component
  - Complex & huge (millions(!) of lines of code)
  - Larger size makes it hard to maintain
- E.g. Multics, Unix, BSD, Linux

Microkernels

- Minimalist approach
  - IPC, virtual memory, thread scheduling
- Put the rest into user space
  - Device drivers, networking, file system, user interface
- More stable with less services in kernel space
- Disadvantages
  - Lots of system calls and context switches
- E.g. Mach, L4, AmigaOS, Minix, K42

Monolithic Kernels VS Microkernels



Hybrid Kernels

- Combine the best of both worlds
  - Speed and simple design of a monolithic kernel
  - Modularity and stability of a microkernel
- Still similar to a monolithic kernel
  - Disadvantages still apply here
- E.g. Windows NT, NetWare, BeOS

Exokernels

- Follows end-to-end principle
  - Extremely minimal
  - Fewest hardware abstractions as possible
  - Just allocates physical resources to apps
- Disadvantages
  - More work for application developers
- E.g. Nemesis, ExOS
- Next Thursday!

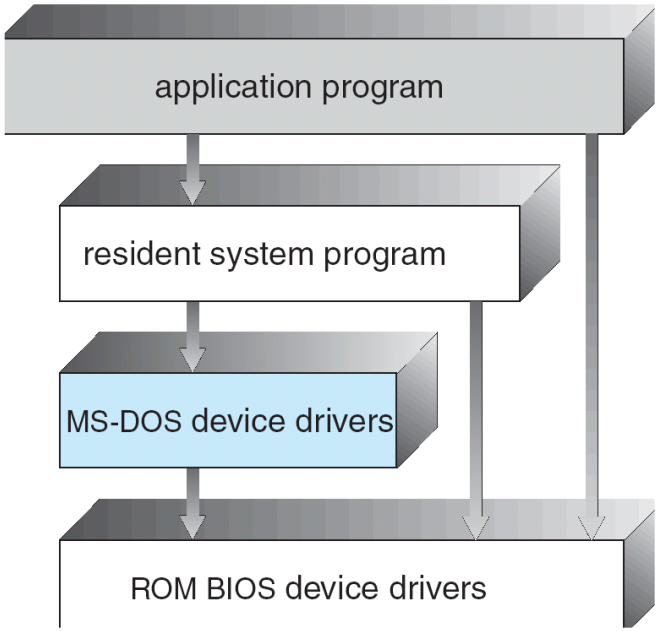
Structuring OS (Kernel)

OS Structure

- Simple Structure (MSDOS)
- Layered Approach
- Microkernel Approach
- Modules Approach
- Client Server Approach

Simple Structure

- MS-DOS – written to provide the most functionality in the least space
  - Not divided into modules
  - Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated



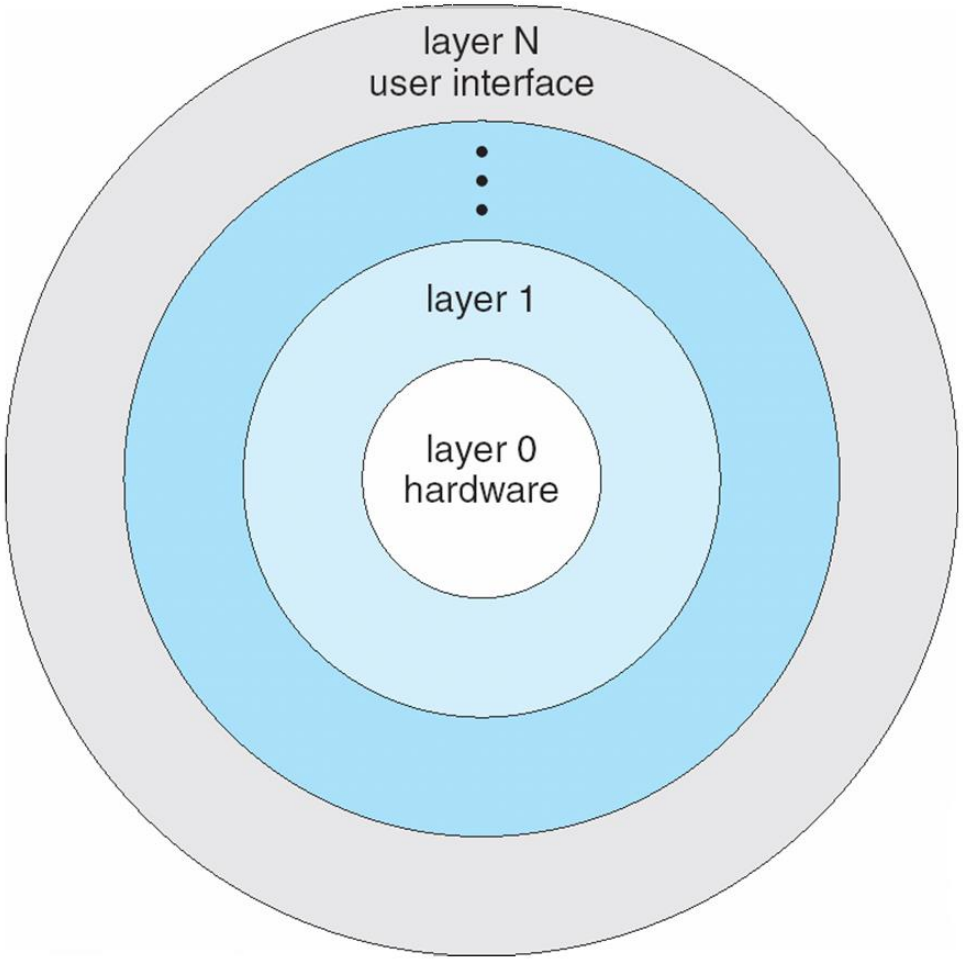
Layered Approach

- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.

- With modularity, layers are selected such that each uses f[illegible]ers

Layer	Function
5	The operator
4	User programs
3	Input/output management
2	Operator-process communication
1	Memory and drum management
0	Processor allocation and multiprogramming

Layered Operating System



Microkernel System Structure

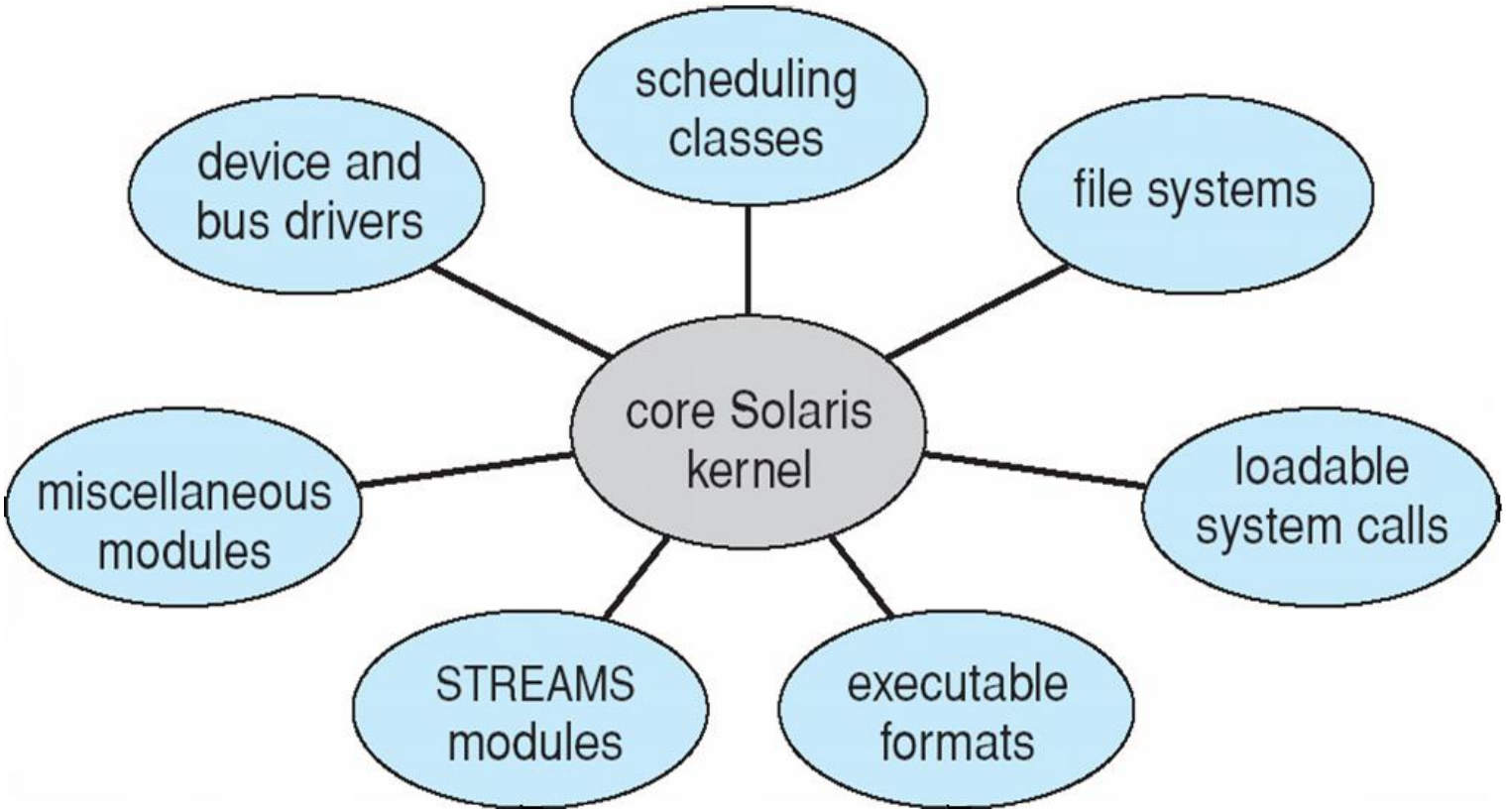
- Moves as much from the kernel into "*user*" space
- Communication takes place between user modules using message passing
- Benefits:
  - Easier to extend a microkernel
  - Easier to port the operating system to new architectures
  - More reliable (less code is running in kernel mode)
  - More secure
- Detriments:
  - Performance overhead of user space to kernel space communication

Modules

- Most modern operating systems implement kernel modules
  - Uses object-oriented approach
  - Each core component is separate
  - Each talks to the others over known interfaces
  - Each is **loadable** as needed within the kernel
- Overall, similar to layers but with more flexible

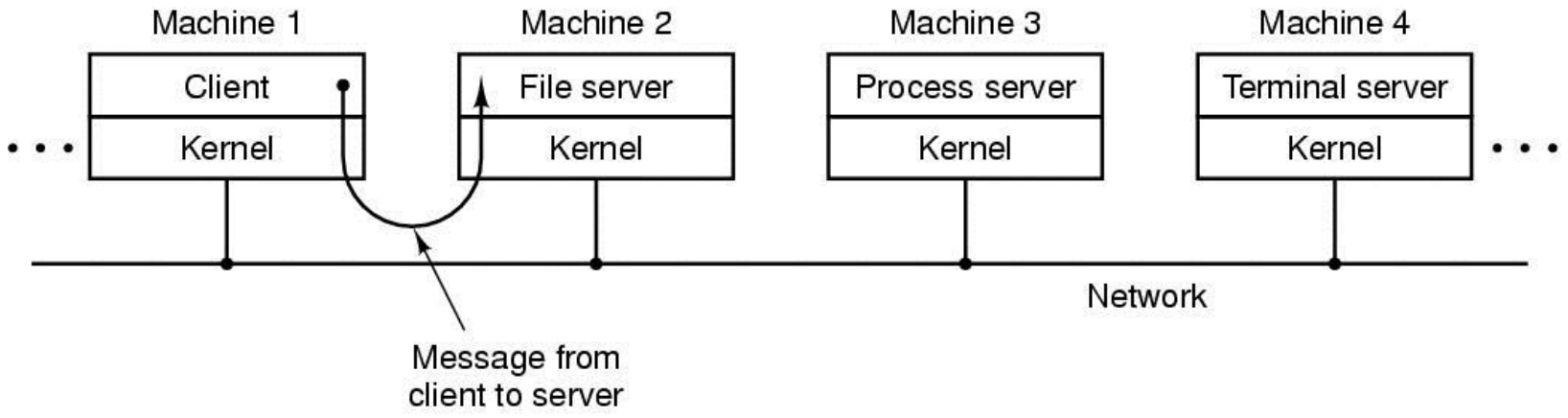
- Linux supports modules

Solaris Modular Approach



Client-Server Model

- Client-Server Model
  - Dumb terminals supplanted by smart PCs
  - Many systems now **servers**, responding to requests generated by **clients**
    - **Compute-server** provides an interface to client to request services (i.e., database)
    - **File-server** provides interface for clients to store and retrieve files



Virtual Machines

- Hardware is abstracted into several different execution environments
  - Virtual machines
- Each virtual machine provides an interface that is identical to the bare hardware
- A *guest* process/kernel can run on top of a virtual machine.
  - We can run several operating systems on the same *host*.
  - Each virtual machine will run another operating system.

Virtual Machines

